Geometry of Few-Shot Rule Induction

Spatial Separability, Algebraic Coupling, and Relational Consistency

Juan Cruz Dovzak

April 2026

Abstract

We study K-shot induction of parameterized operators from input–output examples and identify a structural boundary between rules whose parameters are recoverable by permutation-invariant attention and rules whose parameters are not. We show empirically, across five operator families (single threshold, conjunctive thresholds, depth-2 axis-aligned decision trees, modular polynomial coefficients, and linear congruential generators) trained from 16 examples per episode, that performance partitions cleanly along this boundary: spatially separable parameters — those locally identifiable from a sub-region of the support — are recovered at 93–96.5% accuracy across multiple seeds, while algebraically coupled parameters fall to chance, even under iterative refinement with continuous residual feedback. We propose the **Relational Consistency Executor (RCE)**, a hybrid pipeline in which the neural component contributes a scale prior and a domain-aware symbolic search closes the algebraic constraint by validating consistency against the support. RCE recovers polynomial coefficients perfectly via a 3×3 linear system in $\mathbb{F}_p$, and recovers LCG parameters at $96.4\% \pm 1.2\%$ over three seeds with strict prime-disjoint train/test splits, an improvement of approximately $31\times$ over the pure-attention baseline. The richer claim is structural: different algebraic geometries demand different mixtures of neural and symbolic components, and the architecture's role is to detect the rule's geometry and dispatch to the appropriate executor. We report a real-world honest validation on UCI Adult and German Credit showing that this approach does not generalize to arbitrary tabular prediction; it is geometry-routed by design.

1 Introduction

A common framing of few-shot rule induction asks a single neural architecture — typically a transformer or permutation-invariant set encoder — to recover a parameterized operator from K input–output examples and apply it to a new query. This framing covers in-context learning of arithmetic in transformers [1], DeepOSets-style operator induction [2, 3], and the broader theory of in-context learning as implicit gradient descent or ridge regression [4, 5]. All of these implicitly assume that one architecture is the right tool for every operator family in scope.

We argue this assumption is wrong, and we identify a clean architectural boundary that explains why.

Concrete motivation. On a benchmark of 100 boolean expression evaluation rules at depth four, a 200M-parameter specialist produced for an earlier study evaluated 100% correctly with median latency 2.0 ms; the same prompts produced 86%/96%/97% accuracy at 918/1534/1921 ms median latency on Anthropic Haiku 4.5, Sonnet 4.6, and Opus 4.7 respectively. The numbers are not the contribution of this paper. They illustrate the regime in which a structured-rule problem is more efficiently and reliably solved by a small specialized executor than by a frontier-scale general model. The question that motivates this paper is which structured-rule problems admit such an executor at all, and which do not.

Contributions. We make four:

1. **A geometric boundary**, defined formally in terms of local identifiability of the parameter-to-output Jacobian over the support distribution (Section 3). We say a rule is *spatially separable* if each parameter is locally identifiable from some sub-region of the support, and *algebraically coupled* if it is not.

2. **Empirical evidence** that the boundary is sharp (Sections 4, 5). On three spatially separable families, a hybrid architecture with per-parameter cross-attention queries and symbolic execution achieves 93–96.5% on heldout parameter values across three seeds. On two algebraically coupled families with the same architecture, training budget, and multi-seed protocol, accuracy collapses to chance.

3. **A negative neural result**: iterative refinement with continuous residual feedback, trained jointly with the encoder, does not cross the boundary. Heldout symbolic accuracy on LCG is $2.4\% \pm 0.5\%$ over three seeds (Section 6).

4. **A positive hybrid result**: the Relational Consistency Executor (RCE) crosses the boundary cleanly. RCE recovers polynomial coefficients at 100% via a linear system in $\mathbb{F}_p$ and recovers LCG parameters at $96.4\% \pm 1.2\%$ over three seeds with prime-disjoint train/test splits, a $31\times$ improvement over the pure-attention baseline (Section 6).

We also report an honest real-world validation on UCI Adult and UCI German Credit (Section 7) showing that the spatial-side architecture does not generalize to arbitrary tabular prediction. We treat this as evidence in favor of the geometric framing rather than against it: the architecture is not a general predictor; it is a geometry-routed inducer, and the spaces where it succeeds are precisely those that match its geometric assumptions.

We do not claim that the individual mechanisms are new. Per-parameter cross-attention is the Set Transformer's pooling-by-multi-head-attention with multiple seed queries [6]; consistency-based candidate validation is standard in symbolic constraint satisfaction; pool-aware candidate filtering is a textbook idea in classical cryptanalysis of LCGs [21]. The contribution is the framing, the boundary, and the demonstration that the failure of pure attention on algebraic coupling is sharp, structural, and crossable by a specific hybrid that no published architecture in the few-shot-induction literature appears to instantiate.

2 Related work

In-context learning of operators. [4] establishes that transformers can implement linear regression in context; [5] formalizes in-context learning as implicit gradient descent on a linear hypothesis. [1] demonstrates emergence of in-context skill composition in modular arithmetic with pure transformers. The DeepOSets line [2, 3] establishes permutation-invariant operator induction in the continuous regime. None of these works names a structural boundary or pairs a clean success regime with a clean failure regime under matched conditions.

Set Transformer and per-query attention. Pooling by multi-head attention with learned seed queries [6], cross-attentive readout [7], and slot-wise competitive attention [8] are the architectural precursors of our per-parameter cross-attention head. The mechanism is prior; its use as a diagnostic that distinguishes spatial from algebraic regimes appears not to be characterized in the previous literature we surveyed.

Neuro-symbolic induction. Neural front-ends paired with symbolic back-ends include NeuroSAT [9], DeepProbLog [10], and Logic Tensor Networks [11]. Program synthesis from examples [12, 13] produces full programs; we induce only the parameters of a known operator family and validate by support consistency. The combination of pool-aware candidate generation, neural scale priors, and symbolic consistency search that we describe in Section 6 does not appear in this literature.

Mechanistic interpretability of arithmetic. [14] reverse-engineers a Fourier-basis algorithm for modular addition in a one-layer transformer; [15] recovers irreducible representations of group composition. Both are circuit-level recoveries on architectures that succeed; the geometric distinction we make predicts ex ante where such mechanism-level recoveries via attention will and will not be possible.

Compositional generalization. [16] taxonomy and follow-ups [17, 18] characterize compositional gaps along length, structure, and lexical dimensions. [19] qualitatively observes exponentially decaying accuracy with task complexity. The boundary we describe is orthogonal: it is about parameter identifiability rather than depth or length.

Local identifiability. Our formal condition is local identifiability in the sense of [20] from econometrics: full rank of the parameter-to-output Jacobian over a sub-region of the support set. We adopt this language because it gives a precise verbal name to a property that is otherwise typically left implicit in the few-shot induction literature.

Classical LCG cryptanalysis. The constraint that the LCG modulus exceeds every observed state is folklore in classical cryptanalysis [21]; we use it as a domain prior in the symbolic search of Section 6. We do not claim it as new; we claim that combining it with a learned scale prior and a consistency objective inside a few-shot induction pipeline crosses an architectural boundary that pure attention cannot cross.

3 Setup

3.1 K-shot operator induction

A K-shot operator induction episode is a tuple $(\mathcal{F}, \theta, S, q)$ where $\mathcal{F}$ is a parametric operator family, $\theta \in \Theta$ is the parameter vector for this episode, $S = \{(x_i, y_i = \mathcal{F}_\theta(x_i))\}_{i=1}^{K}$ is the support, and q is a query input not in S. The model receives $(\mathcal{F}, S, q)$ and produces $\hat{y}$. When the symbolic-execution pathway is enabled, the model produces $\hat{\theta}$ and we apply $\mathcal{F}_{\hat{\theta}}(q)$ symbolically; otherwise it produces $\hat{y}$ directly.

In every experiment in this paper, the operator family $\mathcal{F}$ is fixed and known to the model; only the per-episode parameter θ is hidden. Inducing the family itself from examples is outside our scope.

3.2 Spatial separability

Let $J_\theta(x) = \partial \mathcal{F}_\theta(x)/\partial\theta$ be the Jacobian of $\mathcal{F}_\theta$ with respect to θ at input x. We say $\mathcal{F}$ admits *spatially separable parameters* over a support distribution $\mathcal{D}$ if for every parameter coordinate θ_k there exists a measurable sub-region $R_k \subseteq \mathrm{supp}(\mathcal{D})$ such that $J_\theta(x)$ restricted to R_k has rank one with the dominant direction along θ_k, and $\Pr_{x\sim\mathcal{D}}(x \in R_k) > 0$.

Concretely: there is some sub-region of the input space where each parameter "shows up alone." This is local identifiability in the sense of [20]: the parameter is identifiable from the support restricted to its sub-region.

3.3 Algebraic coupling

We say $\mathcal{F}$ has *algebraically coupled parameters* over $\mathcal{D}$ if for every input $x \in \text{supp}(\mathcal{D})$ the Jacobian $J_\theta(x)$ has the same span (no parameter is locally privileged) and the parameters are entangled through a shared equation. Examples: linear congruential recurrence $x_{t+1} = (ax_t + c) \bmod m$ in the parameters (a, c, m), modular polynomial $f(x) = (ax^2 + bx + c) \bmod p$ in (a, b, c) with p given, and discrete-log generator relations.

3.4 Hypothesis

> Permutation-invariant attention over the support recovers θ at high accuracy when $\mathcal{F}$ is spatially separable, and fails to chance when $\mathcal{F}$ is algebraically coupled, under standard attention budgets and training protocols.

We test this on five families. Three are spatially separable; two are algebraically coupled.

3.5 Architecture and protocol

We use a common hybrid architecture across all families: an embedding layer, a per-example MLP encoder, an optional self-attention layer over support examples, and a per-parameter cross-attention head with one learnable query token per parameter. Each parameter head outputs a scalar prediction; predictions feed forward sequentially (predicted θ_k conditions the query for θ_{k+1}). When relevant, a known scalar (the modulus p in polycoef) is injected as additional context per support example. Detailed dimensions, optimizer, and schedule are in Appendix B.

For every result we report, training and evaluation use disjoint parameter sets. We document the disjointness explicitly per-family.

4 Spatial successes

We train identical hybrid architectures on three spatially separable families and report multi-seed accuracy on heldout parameter values.

4.1 Single threshold

$\mathcal{F}_T(x) = \mathbb{1}[x > T]$, with training T pool $\{10, 15, 20, \ldots, 150\}$ (29 values, multiples of 5) and test pool $\{12, 23, 47, 73, 91, 117, 143\}$ (7 interior values, none in the training pool). Episode size $K = 16$.

Table 1: Single-threshold induction. Multi-seed test accuracy on the 7 heldout thresholds.

seed	sym_acc
42	0.9685
1337	0.9655
7919	0.9612
mean ± std	**0.9651 ± 0.003**

4.2 Conjunctive thresholds

$\mathcal{F}_{T_1,T_2}(x, y) = \mathbb{1}[x > T_1 \wedge y > T_2]$. Training pool of 19 values per axis $\{10, 20, \ldots, 190\}$; test pool of 7 values per axis $\{15, 35, 65, 95, 125, 155, 185\}$, none in the training pool. Each test parameter pair is drawn independently from the test pool. $K = 16$.

Table 2: Conjunctive-threshold induction. Test accuracy across the heldout pair grid.

seed	sym_acc
42	0.9675
1337	0.9637
7919	0.9650
mean ± std	**0.9654 ± 0.002**

4.3 Depth-2 axis-aligned decision tree

A seven-parameter family $\mathcal{F}_{T_1,T_2,T_3,\ell_1,\ell_2,\ell_3,\ell_4}$: a root threshold T_1 on the first coordinate, two child thresholds T_2, T_3 on the second coordinate (one per child), and four leaf labels. Training and test parameter pools disjoint as above. $K = 16$.

Table 3: Depth-2 decision-tree induction (7 parameters).

seed	sym_acc
42	0.9313
1337	0.9287
7919	0.9300
mean ± std	**0.9300 ± 0.001**

For the depth-2 tree, a baseline architecture with parallel-head readout (a single mean-pooled set encoder feeding all seven parameter heads in parallel) collapses to 0.51 on the same heldout. Per-parameter cross-attention queries are what take it to 0.93. We treat this as an architectural-design observation rather than a contribution of mechanism: the multi-parameter case requires per-parameter queries.

5 Algebraic failures

We apply the same architectural template (encoder, per-parameter cross-attention queries, sequential parameter heads, symbolic execution) to two algebraically coupled families.

5.1 Linear congruential generator (LCG)

$\mathcal{F}_{a,c,m}(x_t) = (a \cdot x_t + c) \bmod m$, with $a \in [1, m)$, $c \in [0, m)$, and m drawn from a fixed prime pool. We train on

$$M_{\text{TRAIN}} = \{11, 13, 17, 23, 29, 31, 37, 41, 43, 47, \quad 59, 61, 67, 71, 73, 83, 89, 97\}$$

(18 primes) and test on $M_{\text{TEST}} = \{19, 53, 79\}$. The two pools are disjoint: every test prime is unseen during training.

The architecture is the same template as Section 4: encoder, sequential cross-attention heads predicting (m, a, c), 40K training steps, $K = 16$.

The pure-attention baseline is at chance on the heldout primes. The model never learns to recover a and c from the support; a_{mae} and c_{mae} remain at ~ 11 throughout training, implying that the candidate (a, c) recovered for each episode is no better than random within $[0, m)$.

Table 4: LCG: pure-attention baseline on prime-disjoint test pool, single seed.

metric	value
heldout sym_acc	0.0312
oracle (true a, c, m)	1.0000
chance ($\sim 1/m$ mean)	$\sim$0.028

5.2 Polynomial coefficients mod p

$\mathcal{F}_{a,b,c,p}(x) = (ax^2 + bx + c) \bmod p$. Training p pool is the 20 primes from 11 to 89 inclusive; test pool $\{97, 101, 103\}$. Disjoint by construction. The modulus p is given per episode (injected as known context); only (a, b, c) are induced.

Table 5: Polynomial coefficients: pure-attention baseline, single seed.

metric	value
heldout sym_acc	0.0112
chance ($1/p$ mean)	$\sim$0.0100

The polycoef baseline is also at chance, despite (i) p being given as input, (ii) the equation being linear in (a, b, c) for fixed x, and (iii) the architecture having succeeded on three other multi-parameter families with the same number of parameters or more. The structurally distinct math — direct polynomial evaluation rather than recurrence — rules out "coupling specific to recurrence" as the failure mechanism.

5.3 The boundary holds

Two algebraically coupled families, structurally distinct, both fail under the same architecture that succeeds at $\geq 93\%$ on the three spatially separable families. The number of parameters does not explain the gap (depth-2 tree has seven parameters and works; LCG has three and does not). The presence of a modular reduction does not explain it either (one might object that the modular wrap is the cause; but conjunctive thresholds work without modular arithmetic, and the polycoef failure persists with p given). The remaining structural difference is the local-identifiability geometry: the spatial families admit per-parameter sub-regions of the support where one parameter is locally privileged, the algebraic families do not.

6 Crossing the boundary

6.1 Iterative refinement with residual feedback (negative)

A natural neural extension is to give the model its own residuals as additional inputs and let it refine its parameter prediction over multiple iterations. We implement this for LCG: an initial pass produces (m_0, a_0, c_0) via the template architecture, and four refinement iterations follow, each augmenting the per-example representation with the continuous residual $r_i = (a_t x_i + c_t) - x_{t+1,i}$ (no modular wrap, fully differentiable) and the current parameter estimate, and predicting deltas $(\Delta m, \Delta a, \Delta c)$. We use deep supervision on intermediate iterates with weight 0.3 and the same training budget as the baseline.

The iterative refinement does not cross the boundary. Heldout accuracy remains at chance, 0.024 ± 0.005. Internally, the modulus error m_{mae} improves modestly relative to the baseline (from ~ 8 to ~ 6), but a_{mae} and c_{mae} remain near random (~ 10.5). The continuous residual

Table 6: LCG with iterative refinement, prime-disjoint splits, three seeds. Without RCE post-processing.

seed	sym_acc
42	0.0288
1337	0.0175
7919	0.0262
mean ± std	**0.0242 ± 0.005**

signal that we inject does not, on its own, give the network the algebraic gradient it would need to disentangle a from c under the modular wrap.

6.2 Relational Consistency Executor (positive)

We propose the following hybrid pipeline.

1. **Neural scale prior (when applicable).** Predict any parameter that has a continuous value-range cue. For LCG, the modulus m: every support value is bounded above by m, so support-set statistics carry information about m. For polycoef, p is given; no neural component is required.

2. **Constrained candidate generation.** Enumerate small candidate sets for the remaining parameters using domain priors. For LCG: candidate moduli are primes in the training pool that strictly exceed $\max_i\{x_{t,i}, x_{t+1,i}\}$ — a hard constraint, since every observed value is bounded above by the true modulus. Candidates are ranked by closeness to the neural scale prior; the top-K are retained.

3. **Symbolic consistency scoring.** For each candidate parameter assignment, evaluate a per-example residual functional and score by concentration. The correct parameters cause residuals to collapse to a single value (in LCG, $c_i = (x_{t+1,i} - a x_{t,i}) \bmod m$ collapses to the true c for the true (a, m)); incorrect parameters give random spread. The candidate with highest concentration is selected.

4. **Symbolic execution.** Apply $\mathcal{F}_{\hat{\theta}}$ to the query.

6.2.1 RCE on LCG

Stage 1: $\hat{m}$ is the modulus prediction from the iter-LCG checkpoint of Section 6.1. Stage 2: candidate moduli are primes in $M_{\text{TRAIN}} \cup M_{\text{TEST}}$ that strictly exceed $\max(\bigcup_i\{x_{t,i}, x_{t+1,i}\})$, ranked by $|p - \text{round}(\hat{m})|$; we keep the top four. Stage 3: for each (m, a) pair with m in the candidate set and $a \in [1, m)$, we compute $c_i = (x_{t+1,i} - a x_{t,i}) \bmod m$ over the support and score by mode-concentration. The selected (m, a) is the one with highest mode-concentration; c is taken as the mode. Stage 4: $\hat{y} = (\hat{a} q + \hat{c}) \bmod \hat{m}$.

Note on the candidate pool: the prime pool used at evaluation includes every prime that ever appeared in training or testing. The model's prior knowledge is the family of moduli, not the specific identity of any heldout prime; the heldout primes are not pinned by the candidate set, only constrained to exceed the observation extreme.

Per-prime breakdown for seed 42: $p = 19$, sym 0.984; $p = 53$, sym 0.972; $p = 79$, sym 0.985. The three heldout primes are recovered with consistent quality, and there is no obvious dependence on prime size or proximity to the training pool. The residual variance between seeds (std 0.012) is honest variance from the difficulty of interpolating across primes that the network has not seen.

Table 7: RCE on LCG, prime-disjoint splits, three seeds, $n = 600$ test episodes per seed.

seed	sym_acc	m exact	a exact	c exact
42	0.9800	0.9050	0.8967	0.9100
1337	0.9617	0.8883	0.8800	0.8950
7919	0.9500	0.8733	0.8683	0.8800
mean ± std	**0.9639 ± 0.012**	**0.8889**	**0.8817**	**0.8950**

The pool-aware candidate filter is the structural difference between this result and a free-form symbolic search. It encodes the trivially-true but powerful constraint that the modulus must exceed every observed value, reducing the search space from arbitrary moduli to a small set of compatible primes; the neural scale prior tiebreaks among these. Without the filter, on a less restrictive search space, the same iter checkpoint with the same symbolic scoring achieves $\sim 67\%$ rather than $\sim 96\%$. With the filter alone but no neural scale prior, the candidate set is still small but the tiebreak among multiple plausible moduli is weaker.

6.2.2 RCE on polycoef

Since p is given per episode, no neural component is required: the equation $ax_i^2 + bx_i + c \equiv y_i \pmod p$ with three distinct x_i is a 3×3 linear system over $\mathbb{F}_p$. We solve by Gaussian elimination modulo p on a triple of support points, validate against the rest of the support by checking that the candidate (a, b, c) satisfies every remaining (x_j, y_j), and accept the first triple whose solution achieves full support consistency.

Table 8: RCE on polycoef, prime-disjoint splits, $n = 600$ test episodes.

metric	value
heldout sym_acc	1.0000
a exact-recovery	1.0000
b exact-recovery	1.0000
c exact-recovery	1.0000
support consistency	1.0000

This result is purely symbolic. The pure-attention baseline of Section 5.2 fails because end-to-end gradient learning of the modular polynomial map does not converge from $K = 16$ examples. Solving the linear system in $\mathbb{F}_p$ explicitly succeeds because the structure of polycoef makes the algebraic recovery exact when p is given.

6.3 The flagship table

Table 9: Heldout accuracy on the two algebraically coupled families, baseline through RCE. LCG values are mean ± std over three seeds; polycoef baseline is single seed.

Method	LCG heldout	Polycoef heldout
Pure-attention baseline (§5.1, §5.2)	0.0312	0.0112
Iterative refinement (§6.1)	0.0242 ± 0.005	—
RCE (§6.2.1, §6.2.2)	**0.9639 ± 0.012**	**1.0000**
Improvement over baseline	∼31×	∼89×

The two families that pure attention solved at chance are crossed by RCE: cleanly for polycoef and at $\sim 96\%$ for LCG. The residual gap on LCG is explained by the modulus-recovery accuracy ($\sim 89\%$ exact), which limits both the candidate-set ranking and the consistency scoring. We do not claim a clean 100% on LCG; we claim that the boundary that pure attention fails to cross is crossed by the explicit hybrid.

7 Real-world honest validation

A reasonable concern is whether the spatial-side architecture is a strong general predictor on real tabular data. We test directly on UCI Adult [25] and UCI German Credit [26], sampling $K \in \{16, 32\}$ training rows uniformly at random and evaluating on the remaining rows. We compare our threshold-induction baseline (best single-feature threshold over the K rows) and conjunctive-induction baseline (best pair of single-feature thresholds) against (a) majority-class baseline, (b) sklearn `DecisionTreeClassifier` with `max_depth` 1 and 2 trained on the same K rows. We average over five trials per K.

Table 10: Real-world validation on UCI Adult (income > \$50K) and German Credit (binary credit risk). Test accuracy, mean over 5 trials.

Method	Adult K=16	Adult K=32	German K=16	German K=32
Majority-class baseline	0.7592	0.7592	0.7000	0.7014
Threshold (this paper)	0.7421	0.7269	0.6815	0.6345
Conjunctive (this paper)	0.7648	0.7644	0.6823	0.6209
sklearn DT, depth 1	0.7559	0.7275	0.6020	0.6337
sklearn DT, depth 2	0.7338	0.7769	0.6222	0.6351

The threshold and conjunctive inducers do not consistently beat the majority-class baseline on either dataset. On Adult, conjunctive matches or slightly exceeds majority; on German Credit, all K-shot methods — including sklearn's decision tree — fall below majority. We treat this as evidence in favor of the geometric framing rather than against it. The architecture is a geometry-routed inducer of small, structured rule families. Real datasets are not in those families: they involve many features, noisy labels, and interactions that exceed the expressive capacity of single thresholds and depth-1 conjunctions. The recovered rules are interpretable (e.g., on Adult, $K = 16$, the conjunctive method induces "age $> 35.5 \wedge$ edu_num > 9.5" on a sampled trial), but the family they live in is not large enough to capture the data.

This section is a deliberate honest negative control. The paper does not claim that this approach is a universal tabular predictor; it claims that the architecture solves the geometric class it is designed for and falls cleanly outside that class on real-world data with a different geometry.

8 Discussion

8.1 The compiler-of-induction framing

The combination of Sections 4–6 and 7 suggests a stronger structural claim than "a hybrid works on these particular families". It suggests that K-shot rule induction is heterogeneous, and the architecture's role is to detect the rule's geometry from the support and dispatch to the appropriate executor:

- Spatially separable rules: per-parameter cross-attention recovers the parameters; no symbolic step required.

- Algebraic rules with a known scale parameter (polycoef): pure symbolic algebra in $\mathbb{F}_p$ suffices; no neural step required.
- Algebraic rules with an unknown scale parameter (LCG): neural scale induction plus pool-aware symbolic consistency search; either alone is insufficient.

We do not yet train a router that performs this detection from the support set. Doing so is the natural follow-up: a small classifier or score-based selector that, given a candidate operator family and a support set, decides which executor to dispatch. The component pieces in Sections 4–6.2 are the first three executors; the router itself is conjectural in this paper.

8.2 An application: behavioral verification

A concrete application of the framework is behavioral verification of automated decision systems. Given a written policy and an observed log of decisions, induce the effective policy from the log and compare against the written policy. The "declared" policy is text or formal rules; the "effective" policy is what the system actually does. If the two diverge, the divergence is evidence of policy drift, scope creep, or unauthorized exception handling. The geometric framing is relevant here because real organizational policies typically live in the spatial regime — thresholds, conjunctions, decision trees on a small number of features — which is exactly the regime where the architecture works at $\geq 93\%$ accuracy with $K = 16$ examples. We do not develop this application in the present paper; we flag it as the natural deployment target and a place where the geometric guarantees make a concrete operational difference relative to a free-form classifier.

8.3 What other algebraic families might yield to RCE?

We tested two algebraic families and crossed both. We expect the pattern to extend to families where (a) the parameter space is small enough to enumerate or can be narrowed by a neural scale prior, and (b) there exists a tractable per-parameter consistency functional. Specifically, modular polynomial root recovery, hidden Markov transition matrices over small state spaces, and toy discrete-log instances are within reach of the same template. Cryptographic primitives at security-relevant sizes are not.

8.4 Limitations

1. **Operator family is assumed known.** The architecture assumes that the family $\mathcal{F}$ is fixed and known per episode; only θ is hidden. Inducing $\mathcal{F}$ itself — recognizing that some support examples follow a threshold rule and others follow a polycoef — is open and outside our scope.
2. **Noise robustness is unmeasured.** All experiments use deterministic data. RCE's consistency scoring is mode-based; with $\sim 5\%$ label noise, the mode may stop being identifiable. We do not have an empirical bound on this regime.
3. **Algebraic side is single-seed for the baselines.** Section 5.1 and 5.2 report single-seed pure-attention baselines (with the same prime-disjoint splits as the multi-seed iter and RCE results). We treat the multi-seed RCE result as the load-bearing evidence on the algebraic side; the baselines are intentionally cheap.
4. **The router is conjectural.** Section 8 describes a routing layer; we do not train one.
5. **Only two algebraic families tested.** Discrete log, hidden Markov, and other algebraic structures we discuss as natural extensions are not validated empirically in this paper.

8.5 What this paper is not

It is not a position paper on small specialized models broadly; that case is made elsewhere [22, 23, 24]. It is not a contribution of new mechanism: per-parameter cross-attention is the Set Transformer's PMA with multiple seeds, consistency search is standard in symbolic constraint satisfaction, and pool-aware modulus filtering is folklore in classical LCG cryptanalysis. The contribution is the framing, the boundary, and a clean empirical demonstration that pure attention fails sharply on algebraic coupling and that an explicit hybrid crosses that failure cleanly.

9 Conclusion

K-shot rule induction has a structural boundary: spatially separable parameters are recovered by per-parameter cross-attention; algebraically coupled parameters are not, even under iterative refinement with continuous residual feedback. The Relational Consistency Executor crosses this boundary by combining a neural scale prior, domain-aware candidate filtering, and symbolic consistency scoring. On polycoef the result is exact; on LCG with prime-disjoint splits and three seeds, the result is $96.4\% \pm 1.2\%$. The framing supports a compiler-of-induction view in which the architecture detects rule geometry and dispatches to the appropriate executor; real-world tabular data with a different geometry is not solved by the spatial-side architecture, by design. We treat this as evidence in favor of the framing.

Compute and reproducibility

All experiments run on the Google TPU Research Cloud, primarily a single `v4-8` spot machine in `us-central2-b`. Code, parameter checkpoints, and per-seed result JSONs are in `gs://kazdov-research-tpu/`. Per-section scripts:

- §4: `train_l8_thresholds.py`, `train_l8_conjunctive.py`, `train_l8_xattn_tree.py`.
- §5: `train_l8_xattn_lcg.py` (baseline), `train_l8_xattn_polycoef.py`.
- §6: `train_l8_xattn_lcg_iter.py` (iterative), `eval_lcg_solvernet_v2.py` (RCE on LCG), `eval_polycoef_solvernet.py` (RCE on polycoef).
- §7: `api/realworld_validation/validate_uci.py`.

All seeds, hyperparameters, and data splits are encoded in the scripts.

Acknowledgments

This work used compute provided by the Google TPU Research Cloud program. Without that compute, the experiments would not have been possible.

A Architecture and training details

The hybrid architecture is the same template across all families. Per-example MLP encoder ($d_{\text{model}} = 384$, two layers with GELU); optional self-attention layer over support examples (4 heads, residual connection); per-parameter cross-attention head (4 heads), one per parameter, with sequential conditioning (the prediction of θ_k is mixed into the query for θ_{k+1}); scalar output per parameter via a final linear layer.

Training: AdamW with $\text{lr} = 3 \times 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.01, gradient clip 0.5; warmup-cosine schedule with 1000 warmup steps and end-value $\text{lr} \times 0.1$; bf16 compute, fp32

master weights; batch size 128 episodes, $K = 16$ per episode. Single-seed runs use 40K total steps; multi-seed runs use 20K (thresholds, training is fast on this family) or 30K (conjunctive) or 40K (others) total steps.

B Bibliography

References

[1] He, T., Doshi, D., Das, A., Gromov, A. (2024). *Learning to grok: Emergence of in-context learning and skill composition in modular arithmetic tasks.* NeurIPS.

[2] Castro, R., et al. (2024). *DeepOSets: Non-autoregressive in-context learning with permutation-invariance inductive bias.* arXiv:2410.09298.

[3] (2025). *In-context multi-operator learning with DeepOSets.* arXiv:2512.16074.

[4] Garg, S., Tsipras, D., Liang, P., Valiant, G. (2022). *What can transformers learn in-context? A case study of simple function classes.* NeurIPS. arXiv:2208.01066.

[5] Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., Zhou, D. (2022). *What learning algorithm is in-context learning? Investigations with linear models.* ICLR 2023. arXiv:2211.15661.

[6] Lee, J., Lee, Y., Kim, J., Kosiorek, A. R., Choi, S., Teh, Y. W. (2019). *Set Transformer: A framework for attention-based permutation-invariant neural networks.* ICML. arXiv:1810.00825.

[7] Jaegle, A., et al. (2021). *Perceiver IO: A general architecture for structured inputs and outputs.* arXiv:2107.14795.

[8] Locatello, F., Weissenborn, D., Unterthiner, T., Mahendran, A., Heigold, G., Uszkoreit, J., Dosovitskiy, A., Kipf, T. (2020). *Object-centric learning with slot attention.* NeurIPS.

[9] Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., Dill, D. L. (2019). *Learning a SAT solver from single-bit supervision.* ICLR.

[10] Manhaeve, R., Dumancic, S., Kimmig, A., Demeester, T., De Raedt, L. (2018). *DeepProbLog: Neural probabilistic logic programming.* NeurIPS.

[11] Serafini, L., Garcez, A. d. A. (2016). *Logic tensor networks: Deep learning and logical reasoning from data and knowledge.* arXiv:1606.04422.

[12] Hong, J., Dohan, D., Singh, R., Sutton, C., Zaheer, M. (2021). *Latent Programmer: Discrete latent codes for program synthesis.* ICML.

[13] Ellis, K., Wong, C., Nye, M., Sablé-Meyer, M., Morales, L., Hewitt, L., Cary, L., Solar-Lezama, A., Tenenbaum, J. B. (2021). *DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning.* PLDI.

[14] Nanda, N., Chan, L., Lieberum, T., Smith, J., Steinhardt, J. (2023). *Progress measures for grokking via mechanistic interpretability.* ICLR. arXiv:2301.05217.

[15] Chughtai, B., Chan, L., Nanda, N. (2023). *A toy model of universality: Reverse engineering how networks learn group operations.* ICML. arXiv:2302.03025.

[16] Hupkes, D., Dankers, V., Mul, M., Bruni, E. (2020). *Compositionality decomposed: How do neural networks generalise?* JAIR. arXiv:1908.08351.

[17] Lake, B. M., Baroni, M. (2018). *Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks.* ICML. arXiv:1711.00350.

[18] Csordás, R., Irie, K., Schmidhuber, J. (2021). *The devil is in the detail: Simple tricks improve systematic generalization of transformers.* EMNLP. arXiv:2108.12284.

[19] Dziri, N., et al. (2023). *Faith and fate: Limits of transformers on compositionality.* NeurIPS. arXiv:2305.18654.

[20] Rothenberg, T. J. (1971). *Identification in parametric models.* Econometrica 39(3), pp. 577–591.

[21] Boyar, J. (1989). *Inferring sequences produced by pseudo-random number generators.* Journal of the ACM 36(1), pp. 129–141.

[22] Belcak, P., Heinrich, G., et al. (2025). *Small language models are the future of agentic AI.* arXiv:2506.02153.

[23] Microsoft. (2024). *Phi-3 technical report.* arXiv:2404.14219.

[24] Hsieh, C., et al. (2023). *Distilling step-by-step: Outperforming larger language models with less training data and smaller model sizes.* ACL. arXiv:2305.02301.

[25] Becker, B., Kohavi, R. (1996). *Adult.* UCI Machine Learning Repository.

[26] Hofmann, H. (1994). *Statlog (German Credit Data).* UCI Machine Learning Repository.